

ClarifyGPT: Empowering LLM-based Code Generation with Intention Clarification

Overview of Mu et al. (2023)

Presentation by Tali Ratner and Liran Becher

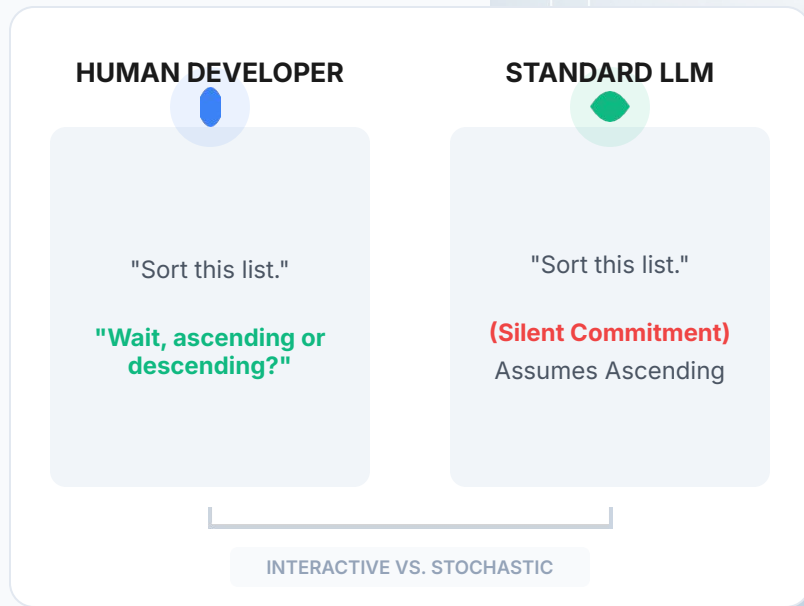


Core Problem

- LLMs often generate code from incomplete or ambiguous prompts
- They usually do not ask users to clarify intent
- Example: "Write a function to sort a list"
- Missing detail: ascending or descending order?
- Result: code may be syntactically correct but semantically wrong

Why Ambiguity Matters in Code Generation

- Real-world requirements are often incomplete
- Developers normally ask follow-up questions
- LLMs silently choose one interpretation
- This creates hidden intent mismatch
- Clarification can improve reliability and user alignment



Two Core Challenges



When to Ask

- Asking too often hurts user experience
- Clear prompts should not be interrupted



What to Ask

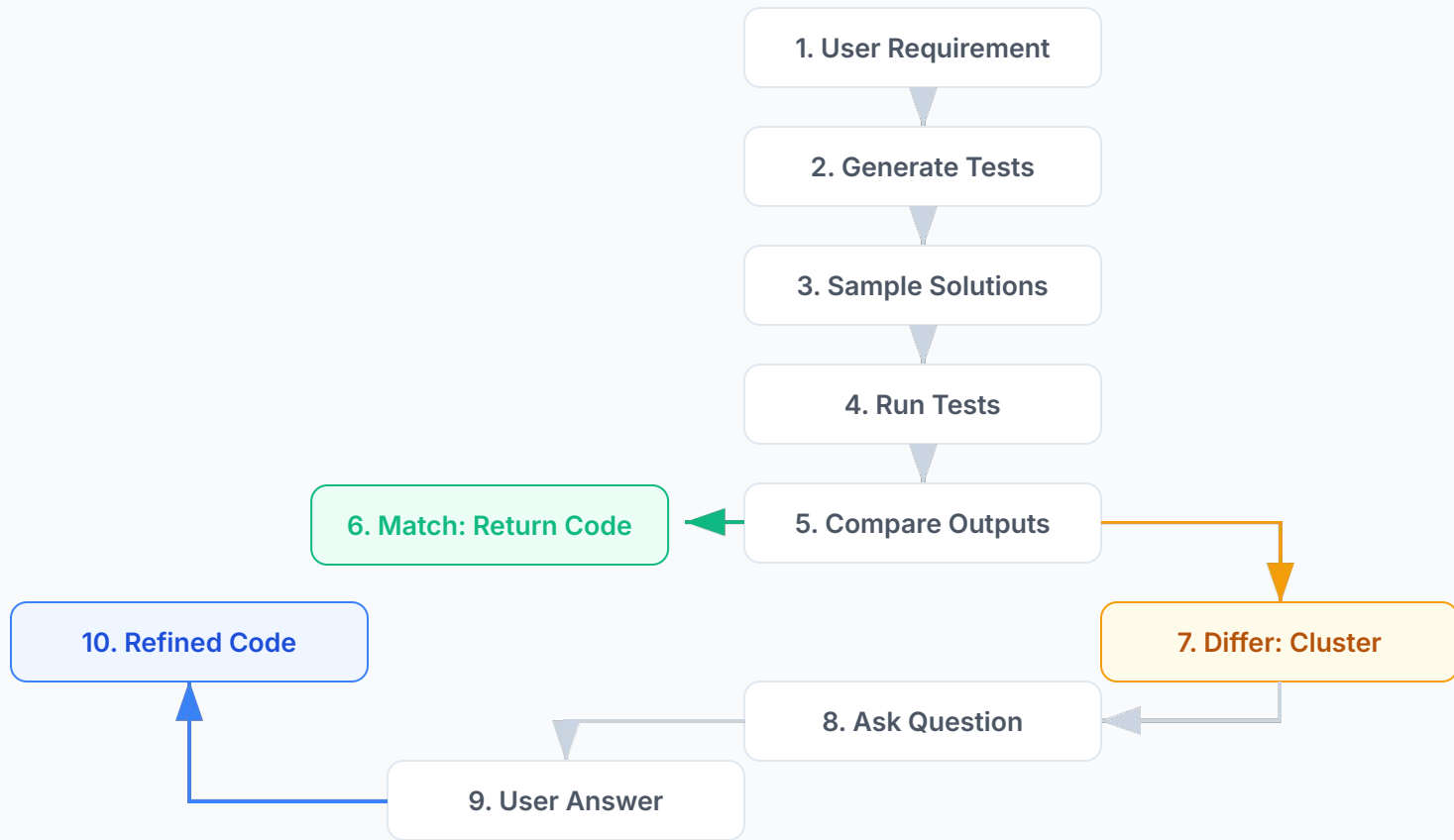
- Questions must target the actual ambiguity
- Vague questions can confuse users

Technical Solution

- ClarifyGPT does not immediately generate final code
- It first creates several draft code solutions
- It tests whether the drafts behave the same way
- If they agree, the prompt is treated as clear
- If they disagree, ClarifyGPT asks a targeted clarifying question



ClarifyGPT Technical Pipeline



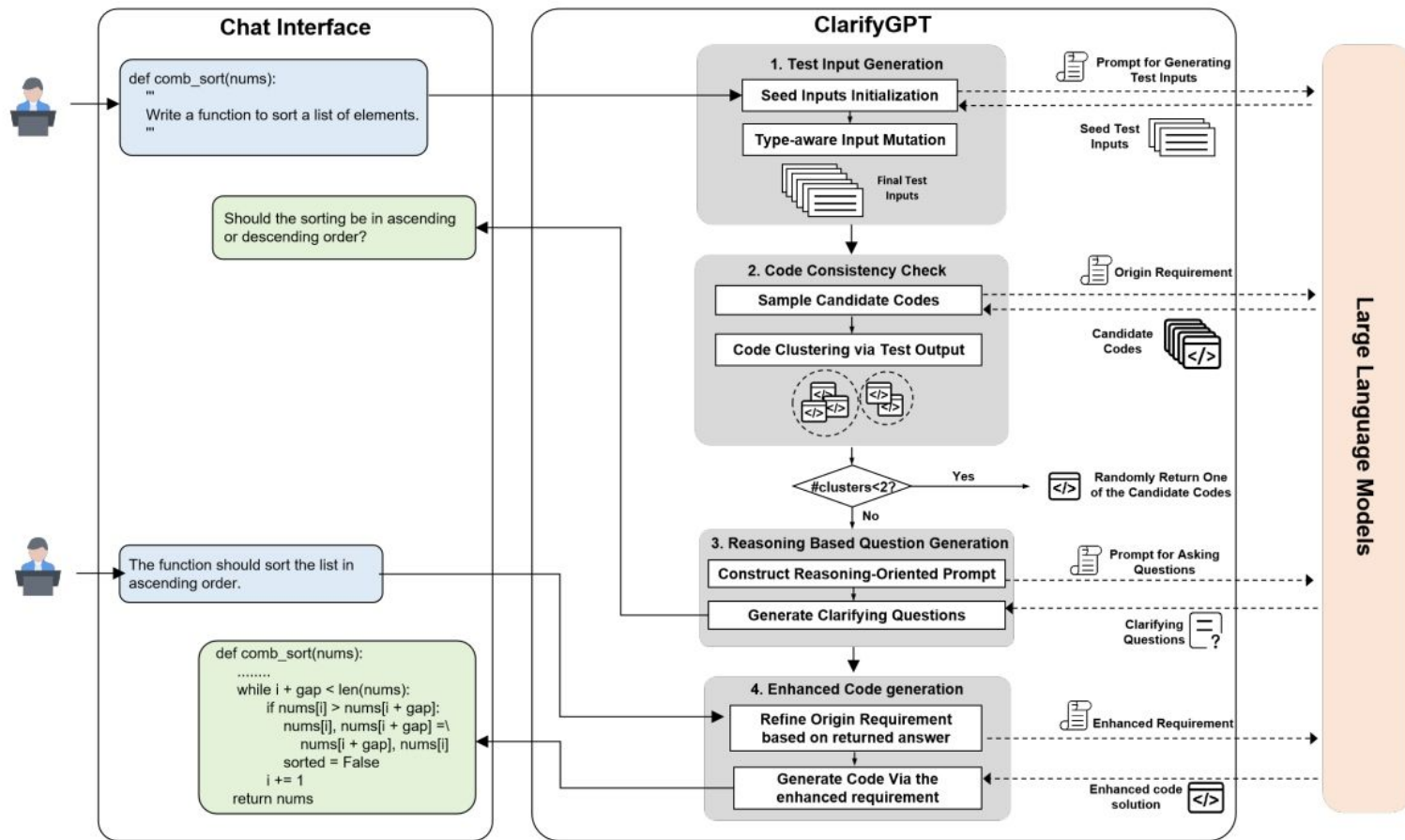


Fig. 1. The Overview of CLARIFYGPT

Evaluation Strategy



Evaluation Modes

- Human feedback
- Simulated user feedback



Benchmarks

- HumanEval
- HumanEval-ET
- MBPP-sanitized
- MBPP-ET



Metric

- Pass@1



Baselines

- Default LLM
- Chain-of-Thought
- GPT-Engineer

Human Evaluation Results



Participant Profile

- 10 participants answered ClarifyGPT's questions
- Diverse group including researchers, students, and developers
- Evaluated targeted clarifying questions against baseline code generation

GPT-4 ON MBPP-SANITIZED

70.96%  **80.80%**

GPT-4 ON MBPP-ET

51.52%  **60.19%**

AVG. RELATIVE IMPROVEMENT

+15.35%

Simulated Feedback Results



Automated Evaluation

- Used LLM-based simulation to scale evaluation
- ClarifyGPT outperformed Default, CoT, and GPT-Engineer
- Key reason: it asks only when ambiguity is detected

GPT-4 AVERAGE PASS@1

68.02%  **75.75%**

CHATGPT AVERAGE PASS@1

58.55%  **67.22%**

AVG. RELATIVE IMPROVEMENT

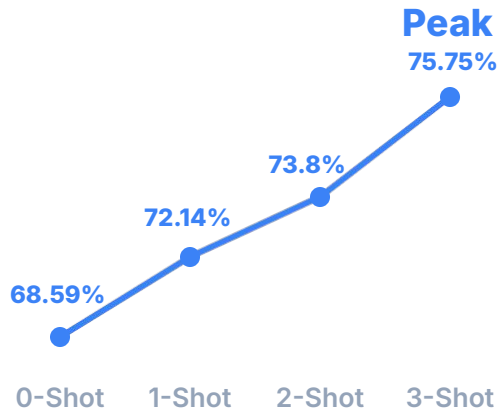
+11.52%

Zero-Shot vs. Few-Shot Prompting

- ClarifyGPT works even with zero demonstrations
- Performance improves as examples increase
- Three-shot prompting gives strongest results
- One-shot prompting may balance performance and efficiency
- Demonstrations help the LLM generalize clarification behavior

PERFORMANCE TREND

Baseline: 68.02%



Critical Critique



Issues & Constraints

- User simulation may be overly optimistic
- Simulated users receive ground-truth test cases
- Human study used only 10 participants
- Participants also saw unit tests to infer intent
- Prompt demonstrations may bias results
- Behavioral agreement may miss subtle ambiguity

Limitations and Edge Cases



No Return Values

Struggles with functions like training loops or UI handlers that produce no output.



Complex Inputs

Difficult to handle file, image, or data streaming inputs in the test suite.



Model Tuning

Requires instruction-tuned LLMs to formulate valid natural language questions.



Subtle Cases

Binary ambiguity detection may miss subtle semantic mismatches.



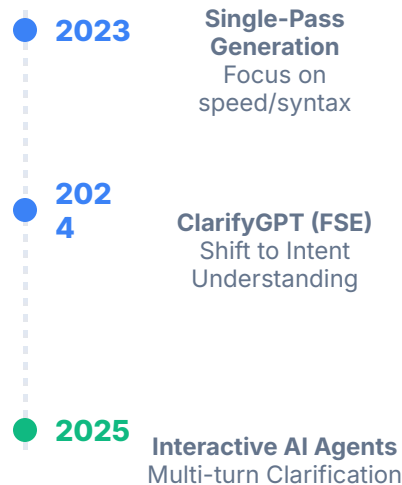
Cost & Latency

Multiple LLM calls significantly increase per-query cost and time.

Why ClarifyGPT Matters Today

- Published in 2023 and formally appeared at FSE 2024
- Arrived as AI coding assistants became mainstream
- Helped shift attention from “generate code fast” to “understand intent first”
- Core idea: LLMs should ask before coding when requirements are unclear
- This idea was standardized since in modern interactive coding tools and agentic coding workflows

EVOLUTION OF CODING AI



Key Takeaways & Future Outlook



INTENT OVER SYNTAX

The primary barrier to AI coding isn't syntax, but the "intent gap" between human requirements and machine interpretation.



SELECTIVE CLARIFICATION

Models achieve higher Pass@1 by asking clarifying questions only when ambiguity is detected, rather than blindly generating code.



AMBIGUITY THROUGH BEHAVIOR

Different outputs from plausible solutions reveal that the same prompt can lead to multiple interpretations.



UNDERSTANDING HAS A COST

Better intent alignment comes with a trade-off: more model calls, more computation, more latency, and more user interaction.